

GANDAKI UNIVERSITY

Faculty of Science and Technology

Bachelor of Computer Application (BCA)

WEB TECHNOLOGY

CHAPTER 4

LEARNING JAVASCRIPT

Complete Theory, Code Examples & Exam Q&A

Prepared by:

Mr. Dammar Khadayat

Lecturer, Department of Science and Technology
Gandaki University, Pokhara, Gandaki Pradesh, Nepal

Academic Year: 2081/82 (2024/25) | Total: 10 Hours

4a. Basics of JavaScript and The Document Object Model

What is JavaScript?

JavaScript (JS) is a lightweight, interpreted, object-oriented programming language primarily used to make web pages interactive and dynamic. It was created by Brendan Eich at Netscape in 1995 and was originally called LiveScript. JavaScript runs directly in the browser (client-side) without needing a server, making web pages respond to user actions instantly.

JavaScript is the third pillar of web development alongside HTML (structure) and CSS (presentation). It adds BEHAVIOR to web pages — responding to clicks, validating forms, animating elements, fetching data from servers, and building complete web applications.

Key Characteristics of JavaScript:

- Interpreted: No compilation needed — browser reads and runs code directly.
- Client-side: Runs in the user's browser, not on the server.
- Event-driven: Code executes in response to user actions (clicks, keystrokes, scrolling).
- Object-based: Everything in JS is treated as an object.
- Case-sensitive: 'myVar' and 'MyVar' are different variables.
- Dynamically typed: Variable types are determined at runtime.
- Platform-independent: Works on all operating systems and browsers.

History of JavaScript:

Year	Milestone
1995	Brendan Eich created JavaScript (then LiveScript) at Netscape in 10 days
1996	Renamed to JavaScript; Microsoft created JScript for Internet Explorer
1997	ECMAScript 1 (ES1) standardized by ECMA International
1999	ECMAScript 3 — introduced try/catch, regular expressions
2009	ES5 — JSON support, strict mode, Array methods
2015	ES6/ES2015 — major update: let, const, arrow functions, classes, promises
2024	ECMAScript 2024 — continues with annual releases of new features

4a.1 Adding JavaScript to HTML

JavaScript can be added to HTML pages in three ways, similar to CSS.

Method 1: Inline JavaScript

```
<!-- Inline JS directly on an HTML element -->
<button onclick="alert('Hello! Welcome to Gandaki
University!')"> Click Me
</button>

<p onmouseover="this.style.color='red'"
onmouseout="this.style.color='black'">
Hover over me!
</p>
```

Method 2: Internal JavaScript

```
<!DOCTYPE html>
<html>
<head>
<title>JavaScript Example</title>
<script>
// Internal JS - inside <script> tags in <head>
function greetUser() {
var name = document.getElementById('nameInput').value;
alert('Welcome, ' + name + '!');
}
</script>
</head>
<body>
<input type='text' id='nameInput' placeholder='Your
name'> <button onclick='greetUser()'>Greet</button>
</body>
</html>
```

Method 3: External JavaScript (Best Practice)

```
<!-- HTML file: index.html -->
<head>
<script src='script.js' defer></script>
</head>

// External file: script.js
function showMessage() {
document.getElementById('output').innerHTML = 'Hello from external
JS!'; }
```

The defer attribute

The defer attribute on <script> tells the browser to download the JS file in the background but only EXECUTE it after the entire HTML document has been parsed. This prevents JS from running before the page elements exist — a very common bug for beginners.

4a.2 The Document Object Model (DOM)

What is the DOM?

The Document Object Model (DOM) is a programming interface (API) that represents an HTML document as a tree of objects. When a browser loads an HTML page, it creates a DOM — a structural, in-memory representation of all elements as nodes in a tree. JavaScript can read and modify this tree to change the page content, style, and structure dynamically.

The DOM is NOT part of JavaScript — it is a Web API provided by the browser. JavaScript uses the DOM to interact with HTML elements. The DOM treats every HTML tag as an object (node) with properties and methods.

DOM Tree Structure Example:

```
HTML Source: DOM Tree:
<html> Document
  <head> └─ <html>
    <title>Page</title> └─ <head>
  </head> └─ <title>
    <body> └─ 'Page' (text node) <h1 id='title'>Hello</h1> └─
  <body>
    <p class='para'>World</p> └─ <h1 id='title'> </body> └─
  'Hello' (text node) </html> └─ <p class='para'> └─ 'World' (text
node)
```

Accessing DOM Elements — Methods:

```
// 1. By ID – most common, returns single element
var heading = document.getElementById('title');

// 2. By Class Name – returns HTMLCollection (array-like)
var paras = document.getElementsByClassName('para');

// 3. By Tag Name – returns HTMLCollection
var allDivs = document.getElementsByTagName('div');

// 4. querySelector – returns FIRST matching element (CSS selector syntax)
var firstBtn = document.querySelector('.btn');
var navLink = document.querySelector('nav a');

// 5. querySelectorAll – returns ALL matching elements (NodeList)
```

Mr. Dammar Khadayat | Gandaki University | Web Technology Notes | Chapter 2 Page 4
[Web Technology — Chapter 4: Learning JavaScript Gandaki University](#)

```
var allBtns = document.querySelectorAll('.btn');
var allLis = document.querySelectorAll('ul li');
```

Modifying DOM Elements:

```
// Get element
var el = document.getElementById('output');

// Change text content (safe – treats as plain text)
el.textContent = 'Hello Gandaki University!';

// Change HTML content (parses HTML tags)
el.innerHTML = '<strong>Welcome!</strong> Click <a href="#">here</a>';

// Change CSS styles
el.style.color = 'red';
el.style.fontSize = '24px';
el.style.backgroundColor = '#FFF3E0';

// Change/add attributes
document.getElementById('myImg').src = 'newphoto.jpg';
document.getElementById('myLink').href = 'https://gandaki.edu.np';

// Add/remove CSS classes
el.classList.add('highlight');
el.classList.remove('hidden');
el.classList.toggle('active');

// Create and add new elements
var newPara = document.createElement('p');
newPara.textContent = 'This paragraph was added by JavaScript!';
document.body.appendChild(newPara);
```

Mr. Dammar Khadayat | Gandaki University | Web Technology Notes | Chapter 2 Page 5
[Web Technology — Chapter 4: Learning JavaScript Gandaki University](#)

4b. Starting to Program with JavaScript — Variables & Operators

4b.1 Variables in JavaScript

What is a Variable?

A variable is a named container used to store data values. In JavaScript, variables can store numbers, text, arrays, objects, boolean values, and more. Unlike strongly-typed languages (Java, C++), JavaScript variables can change type during execution (dynamically typed).

Declaring Variables — var, let, const

```
// — var (old way — function scoped)
var studentName = 'Ram Sharma'; // can be re-declared and
updated var age = 20;
var isEnrolled = true;

// — let (ES6 — block scoped, preferred for mutable
variables) — let courseName = 'Web Technology';
let marks = 85;
marks = 90; // allowed — let variables CAN be updated

// — const (ES6 — block scoped, cannot be reassigned)
const PI = 3.14159;
const UNIVERSITY = 'Gandaki University';
// PI = 3.14; // ERROR: Cannot reassign a const variable!

// — Multiple declarations
let x = 5, y = 10, z =
15;

// — Declaring without value (undefined)
let result;
console.log(result); // Output: undefined
```

var	let / const
Introduced in original JavaScript (ES1)	Introduced in ES6 (2015)
Function-scoped (available in entire function)	Block-scoped (only available within { })
Can be re-declared in same scope	Cannot be re-declared in same scope
Hoisted and initialized to undefined	Hoisted but NOT initialized (Temporal Dead Zone)

var	let / const
Global if declared outside functions	Does not create global window property
Use: legacy code or when you need var hoisting	Use: let for mutable, const for constants

4b.2 Assigning Values and Data Types

```
// — Primitive Data Types
let num = 42; //
Number (integer)
let pi = 3.14; // Number (float/decimal)
let name = 'Ram'; // String (single quotes)
let greeting = "Hello World"; // String (double quotes)
let template = `Hi ${name}!`; // String (template literal -
ES6) let isActive = true; // Boolean
let isDone = false; // Boolean
let nothing = null; // Null - intentional absence of value let
notDefined; // Undefined - declared but no value

// — typeof operator - check data type
console.log(typeof 42); // 'number'
console.log(typeof 'hello'); // 'string'
console.log(typeof true); // 'boolean'
console.log(typeof undefined); // 'undefined'
console.log(typeof null); // 'object' (known JS quirk!)

// — Type Conversion
let numStr =
'25';
let num1 = parseInt(numStr); // '25' → 25 (integer)
let num2 = parseFloat('3.14'); // '3.14' → 3.14 (float)
let str1 = String(100); // 100 → '100' (string)
let str2 = (100).toString(); // 100 → '100' (alternative)
let bool = Boolean(0); // 0 → false
```

4b.3 Lifetime / Scope of a Variable

Variable Scope

Scope refers to the region of code where a variable is accessible (can be used). JavaScript has three types of scope: Global Scope, Function Scope, and Block Scope. Understanding scope is critical to avoiding bugs.

```
// — 1. GLOBAL SCOPE
var globalVar = 'I am global'; // Accessible everywhere

function showGlobal() {
  console.log(globalVar); // Works! ✓
}
```

Mr. Dammar Khadayat | Gandaki University | Web Technology Notes | Chapter 2 Page 7
Web Technology — Chapter 4: Learning JavaScript Gandaki University

```
// — 2. FUNCTION (LOCAL) SCOPE
function calculateMarks() {
  var localVar = 85; // Only accessible inside this function
  console.log(localVar); // Works ✓
}
// console.log(localVar); // ERROR! Not accessible outside ✗

// — 3. BLOCK SCOPE (let and const)
if (true) {
  let blockVar = 'inside if block';
  const MAX = 100;
  console.log(blockVar); // Works ✓
}
// console.log(blockVar); // ERROR! let is block-scoped ✗

// — HOISTING — var declarations are moved to top
console.log(x); // undefined (not an error — var is hoisted)
var x = 5;
// console.log(y); // ERROR — let is NOT hoisted same way
// let y = 10;
```

4b.4 Operators in JavaScript

A. Arithmetic Operators

```

let a = 15, b = 4;

console.log(a + b); // 19 - Addition
console.log(a - b); // 11 - Subtraction
console.log(a * b); // 60 - Multiplication
console.log(a / b); // 3.75 - Division
console.log(a % b); // 3 - Modulus (remainder)
console.log(a ** b); // 50625 - Exponentiation (a to the power b) [ES7]

// Increment and Decrement
let x = 10;
x++; // x = 11 (post-increment: use then increment) ++x;
// x = 12 (pre-increment: increment then use)
x--; // x = 11 (post-decrement)
--x; // x = 10 (pre-decrement)

// Pre vs Post difference:
let n = 5;
console.log(n++); // 5 (returns 5, THEN increments to 6)
console.log(n); // 6
console.log(++n); // 7 (increments FIRST to 7, then returns 7)

```

B. Assignment Operators

Mr. Dammar Khadayat | Gandaki University | Web Technology Notes | Chapter 2 Page 8
[Web Technology — Chapter 4: Learning JavaScript Gandaki University](#)

```

let x = 10;
x += 5; // x = x + 5 = 15 (Add and assign)
x -= 3; // x = x - 3 = 12 (Subtract and assign)
x *= 2; // x = x * 2 = 24 (Multiply and assign)
x /= 4; // x = x / 4 = 6 (Divide and assign)
x %= 4; // x = x % 4 = 2 (Modulus and assign)
x **= 3; // x = x ** 3 = 8 (Exponentiate and assign)

```

C. Comparison / Relational Operators

```

let a = 10, b = '10', c = 20;

// == (loose equality - only checks value, ignores type)
console.log(a == b); // true (10 == '10' - type coercion!)

// === (strict equality - checks BOTH value AND type)
console.log(a === b); // false (number !== string)
console.log(a === 10); // true

// != (loose not equal)
console.log(a != c); // true

// !== (strict not equal)
console.log(a !== b); // true (different types)

console.log(a > 5); // true Greater than
console.log(a < 5); // false Less than
console.log(a >= 10); // true Greater than or equal
console.log(a <= 9); // false Less than or equal

```

== vs === (Critical Distinction)

Always use === (strict equality) in JavaScript. The == operator performs type coercion which causes unexpected results:

0 == false → true, but 0 === false → false

" == false → true, but " === false → false

null == undefined → true, but null === undefined → false

Using === avoids these traps and makes code more predictable and secure.

D. Logical / Boolean Operators

```

let isStudent = true;
let isPaid = false;
let age = 20;

// && (AND) - true only if BOTH conditions are true
console.log(isStudent && isPaid); // false
console.log(isStudent && age >= 18); // true

// || (OR) - true if AT LEAST ONE condition is true

```

```

console.log(isStudent || isPaid); // true
console.log(isPaid || age < 18); // false

// ! (NOT) – reverses the boolean value
console.log(!isStudent); // false
console.log(!isPaid); // true

// Short-circuit evaluation:
// && stops at first false: if (isStudent && isPaid) – if isStudent is
false, isPaid is not checked
// || stops at first true: if (isStudent || isPaid) – if isStudent is
true, isPaid is not checked

```

E. String Operator (Concatenation with +)

```

// + with strings performs CONCATENATION (joining), not
addition let firstName = 'Ram';
let lastName = 'Sharma';
let fullName = firstName + ' ' + lastName;
console.log(fullName); // 'Ram Sharma'

// Mixing string and number:
let age = 20;
let msg = 'Age is: ' + age;
console.log(msg); // 'Age is: 20' (number converted to string)

// CAUTION – unexpected results with +:
console.log('5' + 3); // '53' (string concat, not addition!)
console.log('5' - 3); // 2 (arithmetic – subtraction)
console.log('5' * '2'); // 10 (arithmetic – multiplication)

// ES6 Template Literals (better alternative to +
concatenation) let name = 'Sita';
let marks = 92;
let result = `${name} scored ${marks} out of 100. Grade: A`;
console.log(result); // 'Sita scored 92 out of 100. Grade: A'

```

Conditional Statements, and Looping

4c.1 Functions

What is a Function?

A function is a reusable block of code that performs a specific task. Functions allow you to write code once and call (execute) it as many times as needed, with different inputs each time. Functions are the building blocks of every JavaScript program.

Defining and Calling Functions

```
// — Function Declaration
function greet(name) {
  // 'name' is a PARAMETER
  return 'Hello, ' + name + '!'; // return sends value
  back }

let message = greet('Ram'); // 'Ram' is an ARGUMENT
console.log(message); // 'Hello, Ram!'
console.log(greet('Sita')); // 'Hello, Sita!'

// — Function with multiple parameters
function calculateTotal(price, qty,
discount) {
  let subtotal = price * qty;
  let total = subtotal - discount;
  return total;
}
console.log(calculateTotal(200, 3, 50)); // 550

// — Function with default parameter (ES6)
function greetStudent(name, course =
'BCA') {
  return `${name} is enrolled in ${course}`;
}
console.log(greetStudent('Ram')); // Ram is enrolled in BCA
console.log(greetStudent('Sita', 'MCA')); // Sita is enrolled in
MCA

// — Function Expression
const multiply =
function(a, b) {
  return a * b;
};
console.log(multiply(4, 5)); // 20

// — Arrow Function (ES6) — concise syntax
const add = (a, b) => a + b;
```

```
console.log(add(3, 7)); // 10
```

```
// — Immediately Invoked Function Expression (IIFE) —
```

Mr. Dammar Khadayat | Gandaki University | Web Technology Notes | Chapter 2 Page 11

Web Technology — Chapter 4: Learning JavaScript Gandaki University

```
(function() {  
  console.log('This runs immediately!');  
})();
```

Parameters vs Arguments

Parameters: variables listed in the function DEFINITION. Example: function greet(name) — 'name' is a parameter.

Arguments: actual values passed when CALLING the function. Example: greet('Ram') — 'Ram' is an argument.

A function can accept 0, 1, or many parameters. Extra arguments are ignored; missing ones are 'undefined'.

4c.2 Conditional Statements

A. if / else if / else

```
let marks = 75;  
  
if (marks >= 90) {  
  console.log('Grade: A+ - Distinction');  
} else if (marks >= 80) {  
  console.log('Grade: A - First Division');  
} else if (marks >= 65) {  
  console.log('Grade: B - Second Division');  
} else if (marks >= 45) {  
  console.log('Grade: C - Pass');  
} else {  
  console.log('Grade: F - Fail');  
}  
// Output: Grade: B - Second Division  
  
// — Ternary Operator (shorthand if-else)  
let age = 18;  
let status = (age >= 18) ? 'Adult - Can Vote' : 'Minor - Cannot  
Vote'; console.log(status); // Adult - Can Vote
```

B. switch Statement

```
let day = 3;
let dayName;

switch (day) {
  case 1:
    dayName = 'Sunday';
    break; // IMPORTANT: break stops fall-through to next case
  case 2:
    dayName = 'Monday';
```

Mr. Dammar Khadayat | Gandaki University | Web Technology Notes | Chapter 2 Page 12
Web Technology — Chapter 4: Learning JavaScript Gandaki University

```
break;
  case 3:
    dayName = 'Tuesday';
    break;
  case 4:
    dayName = 'Wednesday';
    break;
  case 5:
    dayName = 'Thursday';
    break;
  case 6:
    dayName = 'Friday';
    break;
  case 7:
    dayName = 'Saturday';
    break;
  default:
    dayName = 'Invalid day number';
}
console.log(dayName); // Tuesday
```

switch — The break Statement

Without break, execution 'falls through' to the next case and continues executing. Always include break at the end of each case unless intentionally using fall-through.

4c.3 Looping / Iteration Statements

A. for Loop — when you know how many times

```
// Syntax: for (initialization; condition; increment/decrement)

// Print 1 to 5
for (let i = 1; i <= 5; i++) {
  console.log('Count: ' + i);
}

// Sum of first 10 natural numbers
let sum = 0;
for (let i = 1; i <= 10; i++) {
  sum += i; // sum = sum + i
}
console.log('Sum:', sum); // Sum: 55

// Looping through an array
let fruits = ['Apple', 'Mango', 'Banana', 'Orange'];
for (let i = 0; i < fruits.length; i++) {
  console.log(i + ': ' + fruits[i]);
}
```

B. while Loop — when condition may become false at any time

Mr. Dammar Khadayat | Gandaki University | Web Technology Notes | Chapter 2 Page 13

[Web Technology — Chapter 4: Learning JavaScript Gandaki University](#)

```
// Syntax: while (condition) { body }

let count = 1;
while (count <= 5) {
  console.log('while count: ' + count);
  count++; // CRITICAL: always update the condition variable!
}

// User input validation (loop until valid input)
let password = '';
while (password !== 'secret123') {
  password = prompt('Enter password:');
}
alert('Access granted!');
```

C. do...while Loop — executes at least ONCE

```
// Syntax: do { body } while (condition);
// The body executes FIRST, THEN condition is checked

let num = 1;
do {
  console.log('do-while: ' + num);
  num++;
} while (num <= 5);

// Practical use: Menu-driven programs
let choice;
do {
  choice = prompt('Enter 1 to continue, 0 to exit:');
  console.log('You chose: ' + choice);
} while (choice !== '0');
console.log('Exited the menu.');
```

D. for...of and for...in Loops (ES6)

```
// for...of – iterates over VALUES of iterable (array,
string) let colors = ['red', 'green', 'blue'];
for (let color of colors) {
  console.log(color); // red, green, blue
}

// for...in – iterates over KEYS/indices of an object
let student = { name: 'Ram', age: 20, grade: 'A' };
for (let key in student) {
  console.log(key + ': ' + student[key]);
  // name: Ram, age: 20, grade: A
}

// break – exits loop immediately
```

```
for (let i = 1; i <= 10; i++) {
  if (i === 5) break;
  console.log(i); // 1, 2, 3, 4
}

// continue – skips current iteration
for (let i = 1; i <= 5; i++) {
  if (i === 3) continue;
  console.log(i); // 1, 2, 4, 5 (3 is skipped)
}
```

for loop	while loop
Best when number of iterations is KNOWN	Best when condition may become false at any time

Initialization, condition, increment in one line	Condition only; init and update placed separately
for (let i=0; i<n; i++)	while (condition)
Loop variable clearly visible	Can be harder to track loop variable
Example: loop through an array	Example: keep asking until valid input given

4d. Events and Event Handling

What is an Event?

An event is any action that occurs in the browser — a user clicking a button, moving the mouse, pressing a key, loading a page, submitting a form, or resizing a window. JavaScript can listen for these events and execute functions (event handlers) in response, making pages interactive.

Common HTML/Browser Events:

Category	Event	Trigger
Mouse	click	User clicks an element
Mouse	dblclick	User double-clicks an element
Mouse	mouseover	Mouse pointer moves over an element
Mouse	mouseout	Mouse pointer moves away from element
Mouse	mousemove	Mouse pointer moves inside element
Keyboard	keydown	Key is pressed down
Keyboard	keyup	Key is released
Keyboard	keypress	Key is pressed and held (deprecated)
Form	submit	Form is submitted
Form	reset	Form is reset/cleared
Form	change	Input value changes
Form	focus	Element receives focus (click or tab)

Form	blur	Element loses focus
Form	input	User types in input field
Window	load	Entire page (including images) has loaded
Window	DOMContentLoaded	HTML fully parsed (before images/CSS)
Window	resize	Browser window is resized
Window	scroll	Page is scrolled

4d.1 Three Ways to Handle Events

Method 1: Inline Event Handler (HTML attribute)

Mr. Dammar Khadayat | Gandaki University | Web Technology Notes | Chapter 2 Page 16

Web Technology — Chapter 4: Learning JavaScript Gandaki University

```
<button onclick="alert('Button was clicked!')">Click Me</button>

<input type='text'
  onkeyup="document.getElementById('output').textContent = this.value"
  placeholder='Type here... '>
<p id='output'></p>

<img src='photo.jpg'
  onmouseover="this.style.opacity='0.5'"
  onmouseout="this.style.opacity='1'">
```

Method 2: DOM Property Event Handler

```
<button id='myBtn'>Click</button>
<script>
  // Assign function to event property
  document.getElementById('myBtn').onclick = function() {
    alert('Button clicked via DOM property!');
  };

  // Arrow function syntax
  document.getElementById('myBtn').onmouseover = () => {
    document.getElementById('myBtn').style.background = 'yellow';
  };
</script>
```

Method 3: addEventListener() — Best Practice

```

<button id='btn'>Click Me</button>
<script>
  const btn = document.getElementById('btn');

  // addEventListener(event, function, useCapture)
  btn.addEventListener('click', function() {
    alert('Clicked via addEventListener!');
  });

  // Named function (can be removed later)
  function handleHover() {
    btn.style.backgroundColor = 'orange';
  }
  btn.addEventListener('mouseover', handleHover);

  // Multiple events on same element
  btn.addEventListener('mouseout', function() {
    btn.style.backgroundColor = '';
  });

  // Keyboard event on document
  document.addEventListener('keydown', function(event) {

```

Mr. Dammar Khadayat | Gandaki University | Web Technology Notes | Chapter 2 Page 17
 Web Technology — Chapter 4: Learning JavaScript Gandaki University

```

    console.log('Key pressed: ' + event.key);
    console.log('Key code: ' + event.keyCode);
  });
</script>

```

Why addEventListener is Best Practice

1. Multiple listeners: You can attach multiple event handlers to the same element and event type.
2. Separation: Keeps JS code separate from HTML.
3. Removable: You can use `removeEventListener()` to detach a named function.
4. More control: Supports event bubbling/capturing with the third parameter. HTML attribute handlers (`onclick='...'`) cannot have multiple handlers and mix HTML with JS.

4d.2 The Event Object

```
// Every event handler receives an event object (often named 'e' or
// 'event') document.addEventListener('click', function(event) {
  console.log(event.type); // 'click'
  console.log(event.target); // element that was clicked
  console.log(event.clientX); // X position of click
  console.log(event.clientY); // Y position of click
});

// Prevent default browser behavior
document.querySelector('a').addEventListener('click',
function(e) { e.preventDefault(); // stops the link from
navigating
  console.log('Link click intercepted!');
});

// Form submission with preventDefault
document.getElementById('myForm').addEventListener('submit',
function(e) { e.preventDefault(); // stops page reload on submit
  console.log('Form submitted via JS!');
});
```

Mr. Dammar Khadayat | Gandaki University | Web Technology Notes | Chapter 2 Page 18

Web Technology — Chapter 4: Learning JavaScript Gandaki University **4e. DOM Event**

Model and Element Positioning

4e.1 Event Propagation — Bubbling and Capturing

Event Propagation

When an event occurs on an element that is nested inside other elements, the event doesn't just fire on that one element — it travels through the DOM tree. This propagation happens in two phases: Capturing (top-down) and Bubbling (bottom-up).

```
// HTML structure:
// <div id='outer'> PHASE 1: CAPTURING
// <div id='inner'> (document → body → outer → inner) // <button
id='btn'>Click</button> ↓
// </div> PHASE 2: TARGET
// </div> ↓
// PHASE 3: BUBBLING
// (btn → inner → outer → body →
document)

// Default: BUBBLING (third param false or omitted)
document.getElementById('outer').addEventListener('click',
function() { console.log('Outer div clicked (BUBBLE)');
}, false);

document.getElementById('btn').addEventListener('click',
function(e) { console.log('Button clicked');
e.stopPropagation(); // Stops bubbling — outer won't
fire });

// CAPTURING phase (third param = true)
document.getElementById('outer').addEventListener('click',
function() { console.log('Outer div clicked (CAPTURE)');
}, true);
```

4e.2 Element Positioning with JavaScript

```
// Getting element dimensions and position
const el = document.getElementById('myBox');

// getBoundingClientRect() — position relative to viewport
const rect = el.getBoundingClientRect();
console.log('Top: ' + rect.top);
console.log('Left: ' + rect.left);
```

```

console.log('Width: ' + rect.width);
console.log('Height: ' + rect.height);

// offsetLeft / offsetTop – position relative to offset
parent console.log(el.offsetLeft); // distance from left of
parent console.log(el.offsetTop); // distance from top of
parent console.log(el.offsetWidth); // element width
including border console.log(el.offsetHeight); // element
height including border

// Setting element position dynamically
el.style.position = 'absolute';
el.style.left = '200px';
el.style.top = '150px';

// Moving element on button click
let posX = 0;
document.getElementById('moveBtn').addEventListener('click',
function() { posX += 50;
  el.style.left = posX + 'px';
});

// Scroll position
window.addEventListener('scroll', function() {
  let scrollPos = window.scrollY;
  if (scrollPos > 300) {
    document.getElementById('backToTop').style.display = 'block';
  }
});

```

4f. Changing Colors and Fonts with JavaScript

JavaScript can dynamically change the visual appearance of elements by modifying their CSS styles through the DOM. This enables real-time interactivity like color pickers, theme switchers, font size controls, and accessibility features.

4f.1 Changing Colors

```
// — Change text color
document.getElementById('title').style.color = '#E67E22';
document.getElementById('title').style.color = 'rgb(230,126,34)';

// — Change background color
document.body.style.backgroundColor = '#1A1A2E';
document.getElementById('card').style.backgroundColor = 'lightyellow';

// — Color Picker — real-time color change
<input type='color' id='colorPicker'
value='#E67E22'>
<p id='sampleText'>Change my color!</p>

<script>
document.getElementById('colorPicker').addEventListener('input',
function() { document.getElementById('sampleText').style.color =
this.value; document.getElementById('sampleText').style.backgroundColor =
this.value + '22';
});
</script>

// — Dark/Light theme toggler
let isDark = false;
document.getElementById('themeBtn').addEventListener('click',
function() { isDark = !isDark;
if (isDark) {
document.body.style.backgroundColor = '#1A1A2E';
document.body.style.color = '#FFFFFF';
this.textContent = 'Switch to Light Mode';
} else {
document.body.style.backgroundColor = '#FFFFFF';
document.body.style.color = '#333333';
this.textContent = 'Switch to Dark Mode';
}
});
```

4f.2 Changing Fonts

```
// — Change font family —
document.getElementById('para').style.fontFamily = 'Georgia, serif';

// — Change font size
document.getElementById('para').style.fontSize = '20px';

// — Font size +/- buttons (Accessibility feature)
<p id='text'>This is sample text for the font size demo.</p>
<button onclick='changeFont(-2)'>A-</button>
<button onclick='changeFont(0)'>Reset</button>
<button onclick='changeFont(2)'>A+</button>

<script>
let currentSize = 16;
function changeFont(delta) {
  if (delta === 0) {
    currentSize = 16; // reset to default
  } else {
    currentSize += delta;
    if (currentSize < 10) currentSize = 10; // min size
    if (currentSize > 36) currentSize = 36; // max size
  }
  document.getElementById('text').style.fontSize = currentSize + 'px';
}
</script>

// — Font weight and style
document.getElementById('heading').style.fontWeight = 'bold';
document.getElementById('quote').style.fontStyle = 'italic';
document.getElementById('price').style.textDecoration = 'line-through';
```

4g. Form Validation with JavaScript

What is Form Validation?

Form validation is the process of checking user input before submitting it to the server. Client side validation with JavaScript provides immediate feedback to users without page reload, improving user experience. It checks: empty fields, correct data formats (email, phone), minimum/maximum length, password strength, etc.

Types of Validation:

- Empty field validation — checking required fields are not blank.

- Length validation — minimum/maximum character count.
- Format validation — email format, phone number pattern, date format.
- Range validation — numeric values within acceptable min/max range.
- Comparison validation — password and confirm password match.
- Custom validation — business rules (e.g., username not already taken).

4g.1 Basic Field Validations

```
// — 1. Empty field check
function
validateEmpty(value, fieldName) {
  if (value === '' || value.trim() === '') {
    alert(fieldName + ' cannot be empty!');
    return false;
  }
  return true;
}

// — 2. String length validation
function
validateLength(value, fieldName, min, max) {
  if (value.length < min) {
    alert(fieldName + ' must be at least ' + min + ' characters.');
```

```
return false;
  }
  if (value.length > max) {
    alert(fieldName + ' must not exceed ' + max + ' characters.');
```

```
return false;
  }
  return true;
}

// — 3. Email validation using Regular Expression
function validateEmail(email) {
  const emailRegex = /^[a-zA-Z0-9.%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$/;
```

```

if (!emailRegex.test(email)) {
    alert('Please enter a valid email address. Example: user@example.com');
    return false;
}
return true;
}

// — 4. Phone number validation (Nepal: 98XXXXXXXX)
function validatePhone(phone) {
    const phoneRegex = /^(98|97)[0-9]{8}$/;
    if (!phoneRegex.test(phone)) {
        alert('Enter valid Nepal phone number (98XXXXXXXX or 97XXXXXXXX)');
        return false;
    }
    return true;
}

// — 5. Numeric range validation
function validateAge(age) {
    age = parseInt(age);
    if (isNaN(age) || age < 16 || age > 100) {
        alert('Age must be a number between 16 and 100.');
```

4g.2 Radio Button and Checkbox Validation

```

// — Radio Button Validation – check if any selected
function validateRadio(radioName) {
    var radios = document.getElementsByName(radioName);
    var selected = false;
    for (var i = 0; i < radios.length; i++) {
        if (radios[i].checked) {
            selected = true;
            break;
        }
    }
    if (!selected) {
        alert('Please select your gender!');
        return false;
    }
    return true;
}

// — Select Box Validation
function validateSelect(selectId, label) {
    var val = document.getElementById(selectId).value;
    if (val === '' || val === '0') {
        alert('Please select ' + label + '!');
```

```

    }
    return true;
}

// — Checkbox Validation — at least one checked
function validateCheckbox(name) {
    var checkboxes = document.getElementsByName(name);
    var checked = false;
    for (var i = 0; i < checkboxes.length; i++) {
        if (checkboxes[i].checked) { checked = true; break; }
    }
    if (!checked) {
        alert('Please select at least one hobby!');
        return false;
    }
    return true;
}

```

4g.3 Complete Form Validation Example

```

<!DOCTYPE html>
<html lang='en'>
<head>
    <title>JS Form Validation – Gandaki University</title>
    <style>
        body { font-family: Arial; max-width: 550px; margin: 30px auto;
background: #f4f4f4; }
        .form-card { background: white; padding: 28px; border-radius: 10px;
box-shadow: 0 3px 12px rgba(0,0,0,0.1); }
        h2 { color: #1A1A2E; text-align: center; margin-bottom: 20px; } label
        { display: block; margin: 12px 0 4px; font-weight: bold; color: #444; }
        input, select, textarea { width: 100%; padding: 9px 12px; border: 1.5px
solid #ccc;
        border-radius: 5px; font-size: 14px; box-sizing: border-box; }
        input:focus, select:focus { border-color: #E67E22; outline: none; }
        .error { color: red; font-size: 12px; margin-top: 3px; display: none; }
        .error.show { display: block; }
        .btn { margin-top: 18px; width: 100%; padding: 11px; background:
#E67E22;
        color: white; border: none; border-radius: 5px; font-size: 16px; cursor:
pointer; }
        .btn:hover { background: #CA6F1E; }
        .success { background: #E8F5E9; padding: 14px; border-radius: 5px;
color: #1B5E20; text-align: center; font-weight: bold; display:
none; }
    </style>
</head>
<body>
<div class='form-card'>
    <h2>Student Registration</h2>
    <form id='regForm'>

```

```

<label>Full Name: *</label>
<input type='text' id='name' placeholder='e.g. Ram Bahadur Sharma'>
<span class='error' id='nameErr'>Name is required (min 3
characters).</span>

<label>Email: *</label>
<input type='email' id='email' placeholder='you@example.com'> <span
class='error' id='emailErr'>Enter a valid email address.</span>

<label>Phone: *</label>
<input type='tel' id='phone' placeholder='98XXXXXXX'> <span
class='error' id='phoneErr'>Enter valid Nepal phone (98/97 + 8
digits).</span>

<label>Gender: *</label>
<input type='radio' name='gender' value='M'> Male
<input type='radio' name='gender' value='F'> Female
<span class='error' id='genderErr' style='display:block;'>Please select
gender.</span>

<label>Province: *</label>
<select id='province'>
<option value=''>-- Select Province --</option>
<option value='Gandaki'>Gandaki</option>
<option value='Bagmati'>Bagmati</option>
<option value='Koshi'>Koshi</option>
</select>
<span class='error' id='provErr'>Please select your province.</span>

<label>Password: *</label>
<input type='password' id='pwd' placeholder='Min 8 characters'>
<span class='error' id='pwdErr'>Password must be at least 8
characters.</span>

<button type='button' class='btn' onclick='validateForm()'>Submit
Registration</button>
</form>
<div class='success' id='successMsg'>
Registration Successful! Welcome to Gandaki University.
</div>
</div>

<script>
function showError(id, show) {
  var el = document.getElementById(id);
  if (show) el.classList.add('show');
  else el.classList.remove('show');
}

function validateForm() {
  let valid = true;

  // 1. Name validation
  let name = document.getElementById('name').value.trim(); if
(name.length < 3) { showError('nameErr', true); valid = false; }

```

```

else showError('nameErr', false);

// 2. Email validation
let email =
document.getElementById('email').value.trim(); let
emailRx = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!emailRx.test(email)) { showError('emailErr', true); valid = false;
} else showError('emailErr', false);

// 3. Phone validation
let phone =
document.getElementById('phone').value.trim(); let
phoneRx = /^(98|97)[0-9]{8}$/;
if (!phoneRx.test(phone)) { showError('phoneErr', true); valid = false;
} else showError('phoneErr', false);

// 4. Gender radio validation
let genders = document.getElementsByName('gender');
let gSelected = Array.from(genders).some(r => r.checked);
showError('genderErr', !gSelected);
if (!gSelected) valid = false;

// 5. Province select validation
let prov = document.getElementById('province').value;
if (!prov) { showError('provErr', true); valid = false; }
else showError('provErr', false);

// 6. Password length validation
let pwd = document.getElementById('pwd').value;
if (pwd.length < 8) { showError('pwdErr', true); valid = false;
} else showError('pwdErr', false);

if (valid) {
document.getElementById('regForm').style.display = 'none';
document.getElementById('successMsg').style.display = 'block';
}
}
</script>
</body>
</html>

```

EXAM QUESTIONS AND ANSWERS — Chapter 4: JavaScript

PART A — Short Answer Questions (2 Marks)

Q: What is JavaScript? List any four features. [2 Marks]

Ans: JavaScript (JS) is a lightweight, interpreted, client-side scripting language used to add interactivity and dynamic behavior to web pages.

Features: (1) Interpreted — runs directly in browser without compilation. (2) Event-driven — responds to user interactions. (3) Object-based — everything is an object. (4) Dynamically typed — variable types are determined at runtime. (5) Case-sensitive — 'Name' and 'name' are different. (6) Platform-independent — runs on all browsers/OS.

Q: What is the DOM? Why is it important for JavaScript? [2 Marks]

Ans: DOM stands for Document Object Model. It is a tree-structured, in-memory representation of an HTML document created by the browser when a page loads. Every HTML element becomes a node in this tree.

Importance: JavaScript uses the DOM to: (1) Access elements (getElementById, querySelector). (2) Modify content (innerHTML, textContent). (3) Change styles (style.color, style.fontSize). (4) Add/remove elements. (5) Respond to events. Without the DOM, JavaScript cannot interact with HTML pages.

Q: What is the difference between var, let, and const in JavaScript? [2 Marks]

Ans: var: Function-scoped. Can be re-declared and updated. Hoisted. Old syntax (ES1). Avoid in modern code.

let: Block-scoped. Cannot be re-declared in same scope but CAN be updated. No hoisting issue. ES6. Use for mutable variables.

const: Block-scoped. Cannot be re-declared OR updated (constant). Must be initialized when declared. ES6. Use for values that should never change.

Q: What is the difference between == and === in JavaScript? [2 Marks]

Ans: == (Loose Equality): Compares VALUES only, performing TYPE COERCION. Example: '5' == 5 → true (string converted to number).

=== (Strict Equality): Compares BOTH value AND data type. No coercion. Example: '5' === 5 → false (string ≠ number).

Best Practice: Always use === in JavaScript to avoid unexpected bugs caused by type coercion.

Q: What is the difference between a for loop and a while loop? [2 Marks]

Ans: for loop: Best when the number of iterations is KNOWN beforehand. Initialization, condition, and increment are all in one line. Example: for (let i=0; i<5; i++) while loop: Best when iterations depend on a condition that may become false at any time. Condition only in the while clause; init and update placed separately. Example: while (userInput !== 'quit') do-while: Like while but executes at least ONCE before checking the condition.

Q: What is an event in JavaScript? Name any five events. [2 Marks]

Ans: An event is any action that occurs in the browser — typically triggered by a user or the browser itself. JavaScript can listen for events and run code (event handlers) in response. Five events: (1) click — element is clicked. (2) submit — form is submitted. (3) keydown — keyboard key is pressed. (4) mouseover — mouse moves over element. (5) load — page finishes loading. (6) change — input value changes. (7) focus — element receives focus. (8) blur — element loses focus.

Q: What is form validation? Why is client-side validation important? [2 Marks]

Ans: Form validation is checking user input before it is sent to the server, ensuring it meets required format, length, and content rules.

Importance of client-side validation: (1) Immediate feedback — user sees errors without page reload. (2) Better UX — faster and more responsive. (3) Reduces server load — invalid data is rejected before reaching server. (4) Catches common errors — empty fields, wrong email format, weak passwords. Note: Client-side validation alone is NOT secure — server-side validation is ALSO required.

Q: What is the difference between innerHTML and textContent? [2 Marks]

Ans: innerHTML: Gets or sets the HTML content of an element, PARSING HTML tags. Example: el.innerHTML = 'Bold' renders bold text.

textContent: Gets or sets only the TEXT content, treating everything as plain text. No HTML parsing. Safer (prevents XSS attacks). Example: el.textContent = 'Bold' displays the literal text 'Bold'.

Use textContent when displaying user-provided data (security). Use innerHTML when you intentionally want to insert HTML.

Q: Explain event bubbling and capturing in JavaScript. [2 Marks] **Ans:** Event

Propagation has two phases: Capturing (top-down) and Bubbling (bottom-up).

Event Capturing: When an event occurs, it first travels DOWN from the document root to the target element. Handlers registered with true as the third parameter fire during this phase. Event Bubbling (default): After the event fires at the target, it BUBBLES UP through parent elements to the document root. Handlers registered with false (or no third param) fire during this phase.

`e.stopPropagation()` stops the event from continuing to bubble/capture. `e.preventDefault()` prevents the default browser action.

Q: What is an arrow function in JavaScript? How is it different from a regular function? [2 Marks]

Ans: Arrow function is a concise ES6 syntax for writing functions. Instead of function keyword, use `=>`.

Syntax: `const add = (a, b) => a + b;` (one-line) or `const greet = (name) => { return 'Hi ' + name; }`

Differences: (1) Shorter syntax. (2) Arrow functions do NOT have their own 'this' — they inherit 'this' from surrounding scope. (3) Cannot be used as constructors. (4) No 'arguments' object. Regular functions are needed when you need their own 'this' binding (e.g., object methods, DOM event handlers in some cases).

PART B — Long Answer Questions (5–10 Marks)

Q: Explain all JavaScript operators with examples: Arithmetic, Assignment, Comparison, Logical, String. [5 Marks] [5 Marks]

Ans: Arithmetic Operators: `+`, `-`, `*`, `/`, `%` (modulus), `**` (exponentiation), `++` (increment), `--` (decrement). Example: `let a=10, b=3; a%b = 1` (remainder); `a**b = 1000`.

Assignment Operators: `=`, `+=`, `-=`, `*=`, `/=`, `%=`. Example: `x+=5` means `x=x+5`. Shorthand for compound operations.

Comparison Operators: `==` (loose equal), `===` (strict equal), `!=`, `!==`, `>`, `<`, `>=`, `<=`. Example: `'5'==5` is true but `'5'===5` is false. Always use `===` to avoid type coercion bugs.

Logical Operators: `&&` (AND — both true), `||` (OR — at least one true), `!` (NOT — reverses). Example: `(age>=18 && hasID)` for voting eligibility.

String Operator: `+` performs concatenation with strings. Example: `'Hello' + ' ' + 'World' = 'Hello World'`. Use template literals (``${var}``) as modern alternative.

Q: Explain JavaScript functions: declaration, expression, arrow functions, parameters/arguments, return, and scope. Write examples. [5 Marks] [5 Marks]

Ans: Function Declaration: `function name(params) { ... }` — can be called before declaration (hoisting). Example: `function greet(name) { return 'Hello ' + name; }`

Function Expression: `const fn = function(params) { ... }` — NOT hoisted. Stored in variable.

Arrow Function (ES6): `const fn = (params) => expression;` — concise, no own 'this'. Example:
`const add = (a,b) => a+b;`

Parameters vs Arguments: Parameters are placeholders in definition; Arguments are values passed at call time.

Return statement: Sends a value back to the caller and exits function. Functions without return give undefined.

Scope: Variables inside a function are LOCAL (not accessible outside). Global variables are accessible inside functions. Block scope with `let/const` inside `{ }`.

Q: Explain all types of loops in JavaScript with syntax, examples, and when to use each. [5 Marks] [5 Marks]

Ans: for loop: `for(init; condition; update) { }` — Use when iterations are KNOWN. Example: print 1-10, loop array.

while loop: `while(condition) { }` — Use when iterations are UNKNOWN (condition-dependent). Example: keep asking until correct password.

do...while loop: `do { } while(condition);` — Executes at LEAST ONCE regardless of condition. Example: menu-driven programs.

for...of loop (ES6): `for(let val of array) { }` — Iterates over VALUES of arrays/strings. Cleaner than index-based for loop.

for...in loop: `for(let key in object) { }` — Iterates over KEYS (property names) of an object. `break`: Exits the loop immediately. `continue`: Skips current iteration and moves to next.

Q: Write a complete JavaScript program for form validation that validates: name (not empty, min 3 chars), email (valid format), phone (Nepal format), gender (radio selected), province (select not empty), and password (min 8 chars). Show error messages inline. [10 Marks] [10 Marks]

Ans: A complete form validation program should include:

1. HTML form structure with inputs for name, email, phone, gender (radio), province (select), password.
2. CSS styling: `.error` class hidden by default (`display:none`), shown with `.show` class (`display:block`). Red color for errors.
3. JavaScript validation function `validateForm()` called on button click.
4. Name: `value.trim().length >= 3`, else show nameErr.
5. Email: test against regex `/^[^\s@]+@[^\s@]+\.[^\s@]+$/` (must have @ and domain).
6. Phone: test against `/^(98|97)[0-9]{8}$/` for Nepal format.
7. Gender: `Array.from(getElementsByName).some(r=>r.checked)` to check any radio selected.
8. Select: check value `!== ''` for province dropdown.
9. Password: `pwd.length >= 8`.
10. Only show success message if ALL validations pass (`valid === true`). Refer to complete code in Section 4g.3 of these notes for the full implementation.

Q: Explain the DOM. How does JavaScript access and modify HTML elements? Write examples for getElementById, querySelector, innerHTML, style modification, and addEventListener. [5 Marks] [5 Marks]

Ans: DOM (Document Object Model) is a tree-structured API that browsers create from HTML. Every tag is a node/object with properties and methods.

1. getElementById('id') — returns single element by its unique id.
2. getElementsByClassName('name') — returns array-like of elements with that class.
3. querySelector('css-selector') — returns FIRST matching element using CSS selector.
4. querySelectorAll('css-selector') — returns ALL matching elements (NodeList).
5. innerHTML — gets/sets HTML content (parses tags). textContent — sets plain text only.
6. style.property — changes CSS: el.style.color = 'red'; el.style.fontSize = '20px';
7. classList.add/remove/toggle — manages CSS classes.
8. addEventListener(event, function) — attaches event listener for interactive behavior.
9. createElement / appendChild — creates and adds new DOM elements dynamically.

PART C — Objective / MCQ Questions

Q#	Question	Answer
1	Who created JavaScript?	Brendan Eich (Netscape, 1995)
2	JavaScript was originally called?	LiveScript
3	JS runs on which side?	Client-side (browser)
4	Which tag embeds JS in HTML?	<script>
5	Which JS method shows a dialog box?	alert()
6	Which JS method shows input dialog?	prompt()
7	What does DOM stand for?	Document Object Model
8	Which method gets element by ID?	document.getElementById('id')
9	Which method gets first matching CSS selector?	document.querySelector('.class')
10	innerHTML vs textContent — which parses HTML?	innerHTML (textContent is plain text)

11	Which keyword declares a constant?	const
12	Which keyword is block-scoped and mutable?	let
13	Which operator checks value AND type?	=== (strict equality)
14	What does % operator return?	Remainder (modulus) of division
15	x++ vs ++x — what is the difference?	x++ returns then increments; ++x increments then returns

Mr. Dammar Khadayat | Gandaki University | Web Technology Notes | Chapter 2 Page 32

Web Technology — Chapter 4: Learning JavaScript Gandaki University

Q#	Question	Answer
16	Which logical operator returns true if BOTH are true?	&& (AND)
17	Which logical operator returns true if ANY one is true?	(OR)
18	What does ! operator do?	Reverses/negates boolean value
19	'5' + 3 in JavaScript gives?	'53' (string concatenation)
20	Template literal syntax uses?	Backticks `` with \${expression}
21	Which statement exits a loop immediately?	break
22	Which statement skips current iteration?	continue
23	do...while runs body at least?	Once (condition checked AFTER first run)
24	Which loop iterates over array VALUES?	for...of loop
25	Which loop iterates over object KEYS?	for...in loop
26	switch case needs which statement to stop fall through?	break
27	Ternary operator syntax?	condition ? valueIfTrue : valueIfFalse
28	Which method adds event listener to element?	addEventListener(event, function)
29	Which event fires when form is submitted?	submit
30	e.preventDefault() does what?	Stops the default browser behavior
31	e.stopPropagation() does what?	Stops event bubbling/capturing
32	Default event propagation is?	Bubbling (bottom-up)
33	Which CSS property is changed by el.style.backgroundColor?	background-color

34	Which regex tests for a valid email?	<code>/^[^@]+@[^@]+\.[^@]+\$/</code>
35	NaN stands for?	Not a Number — result of invalid arithmetic

Chapter 4 — Summary & Quick Revision

Key Points to Remember

- JavaScript = client-side scripting language. Created by Brendan Eich, 1995. Makes pages interactive.
- Add JS: Inline (onclick=""), Internal (<script>), External (<script src='file.js'>). External is best practice.
- DOM = Document Object Model — tree of HTML elements. JS reads/modifies DOM to change pages.
- DOM access: getElementById, querySelector, querySelectorAll. Modify: innerHTML, textContent, style, classList.
- Variables: var (function scope), let (block scope, mutable), const (block scope, immutable). Use let/const.
- Operators: +, -, *, /, % (arithmetic), ===, !==, >, < (comparison), &&, ||, ! (logical), + (string concat).
- Always use === (strict) not == (loose) to avoid type coercion bugs.
- Functions: declaration, expression, arrow (=>). Parameters vs Arguments. return sends value back.
- Conditionals: if/else if/else, switch/case (with break), ternary (? :).
- Loops: for (known count), while (condition-based), do-while (at least once), for-of (array values), for-in (object keys).
- Events: click, submit, keydown, mouseover, load, change. Handlers: inline, DOM property, addEventListener (best).
- Event propagation: Capturing (down) then Bubbling (up). stopPropagation() stops. preventDefault() prevents browser default.
- DOM positioning: getBoundingClientRect(), offsetLeft, offsetTop, style.left/top. • Form validation: empty check (trim), email regex, phone regex, radio (some().checked), select (value !== ""), password length.

